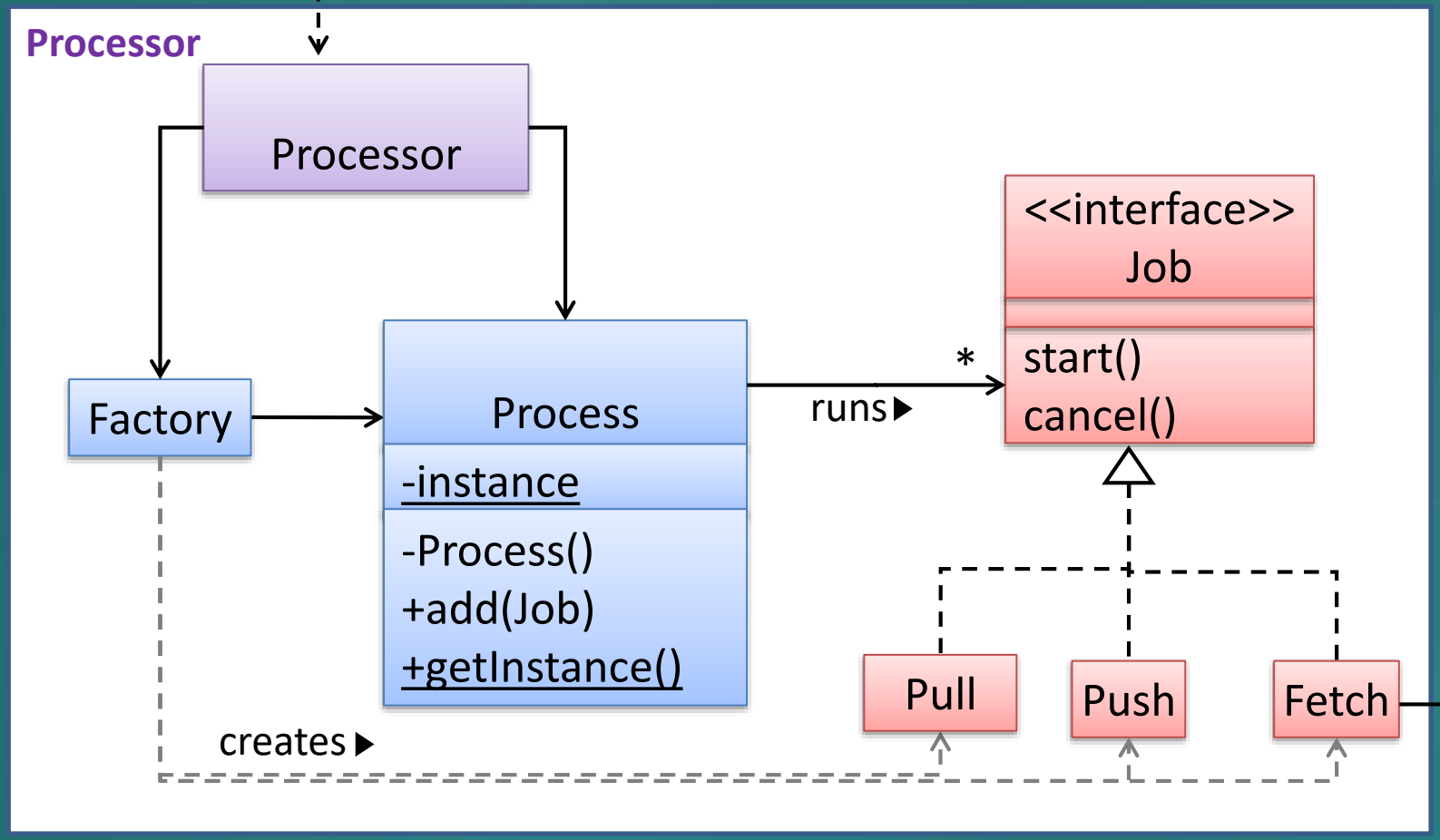
WARNING

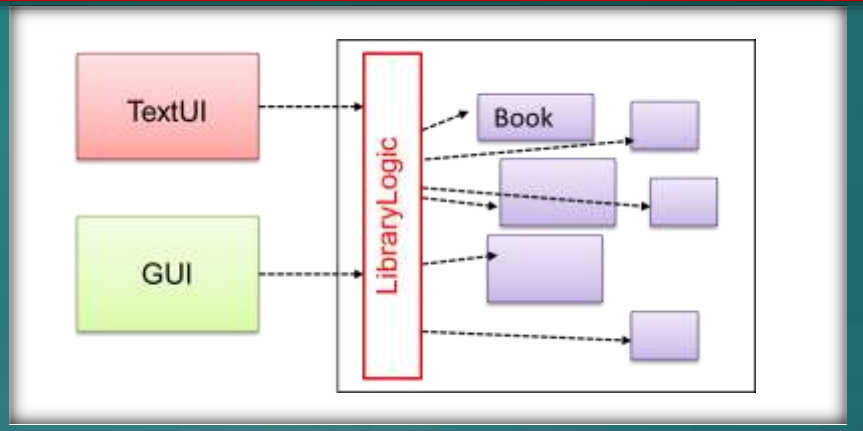
These slides are not optimized for printing or exam preparation. They are for lecture delivery only.

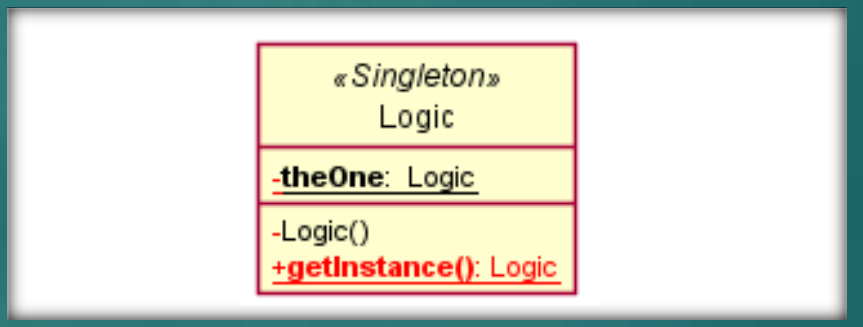
As these slides contain a lot of animations, watch in slideshow mode for optimal results.

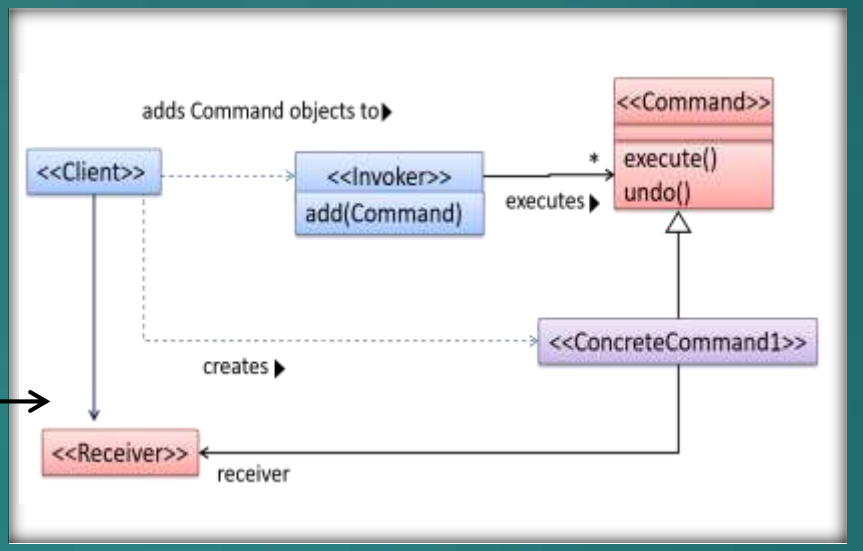
Which of these design patterns are likely to be in the following design?

- ❑ Façade,
- ❑ Singleton
- ❑ Command







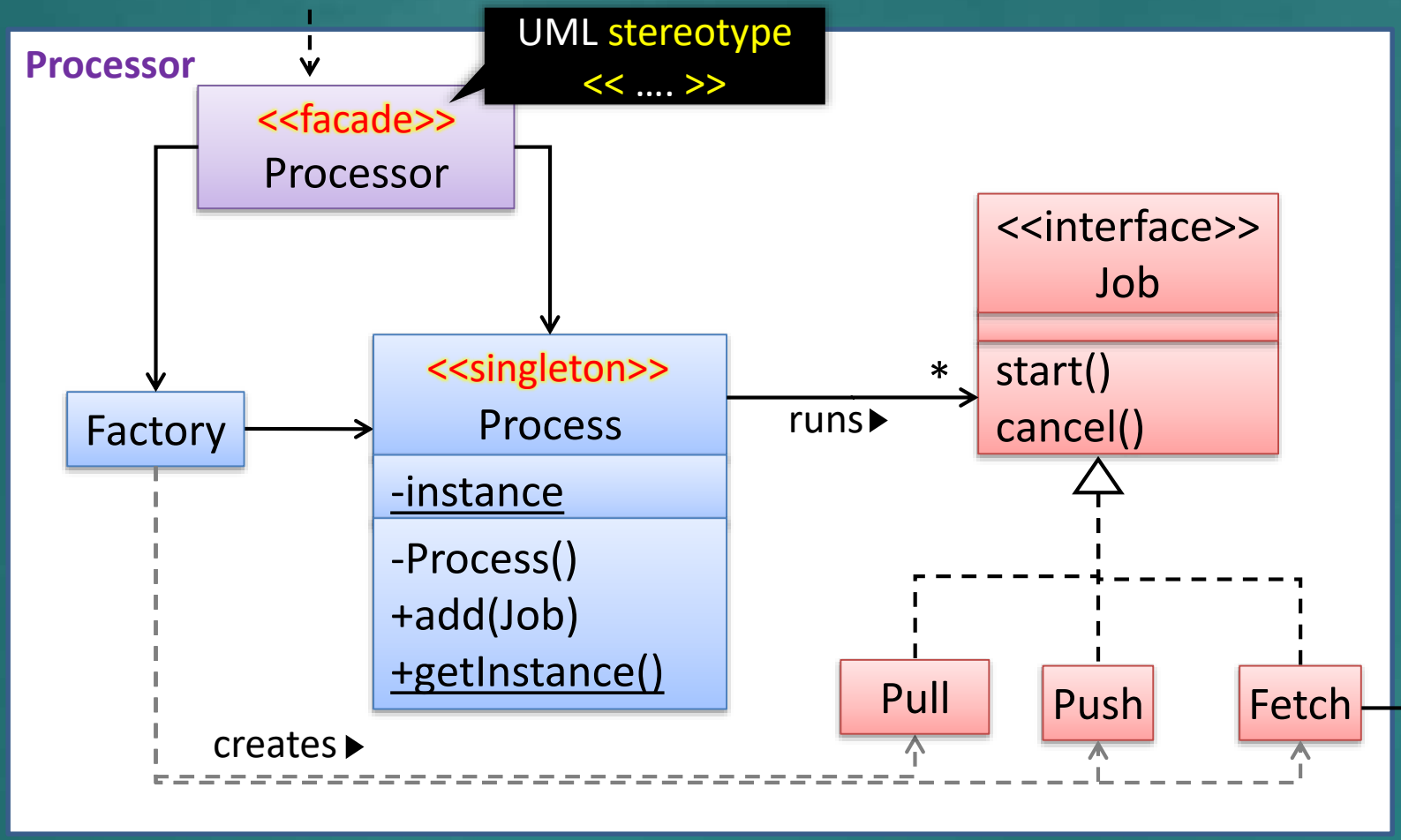


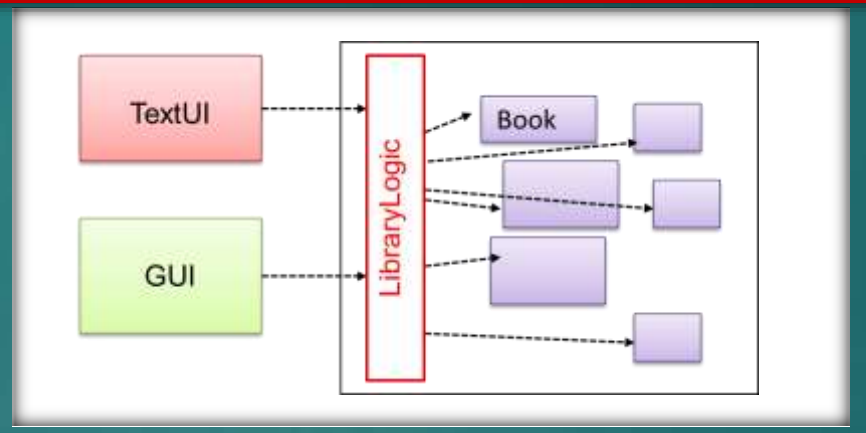
Have a look at the W10→11-weekly-briefing-exercises.pdf (in Canvas/files) if you haven't done so already.

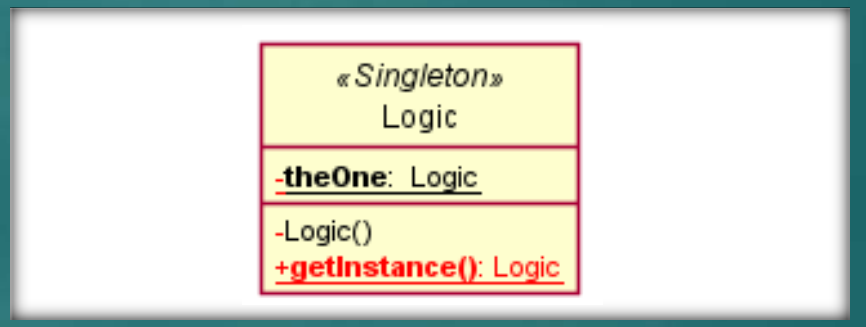
Q1

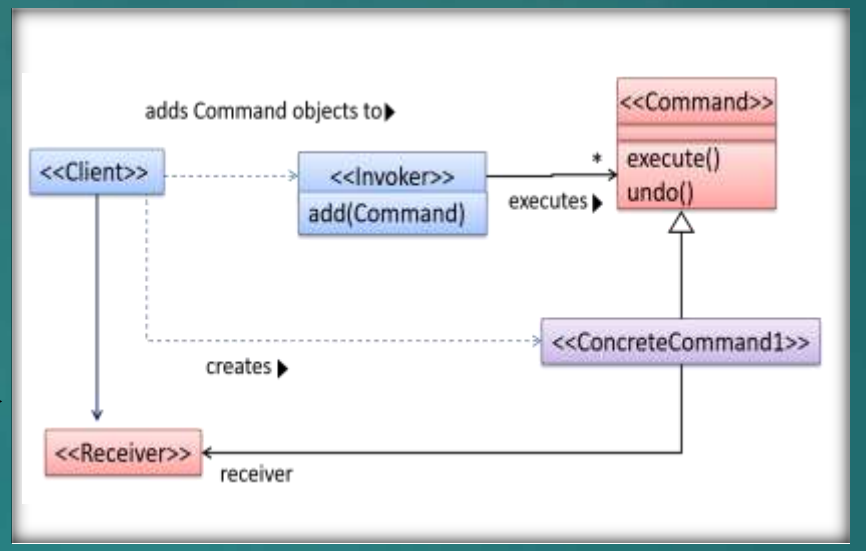
Which of these design patterns are likely to be in the following design?

- ❑ Façade,
- ❑ Singleton
- ❑ Command





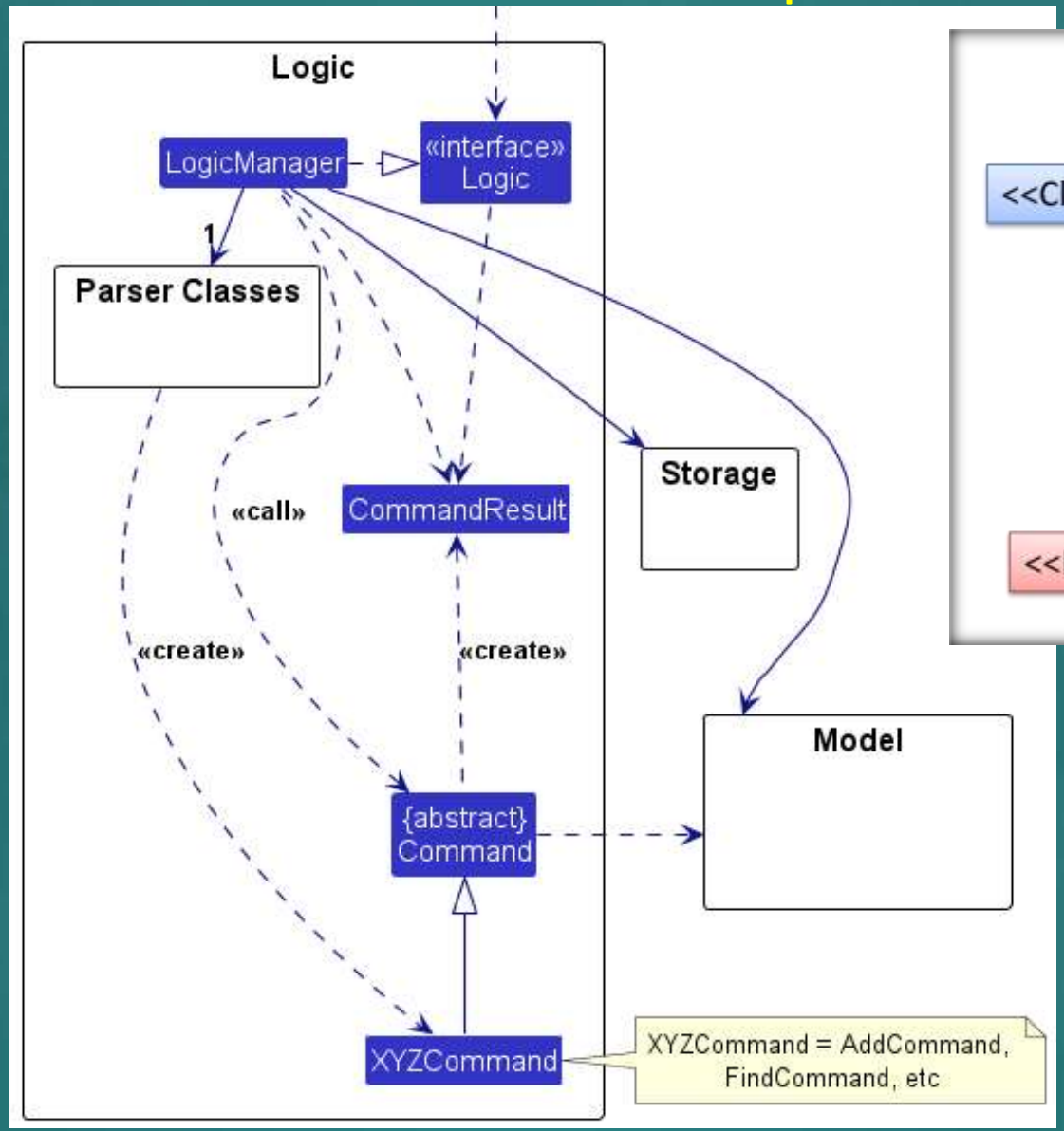


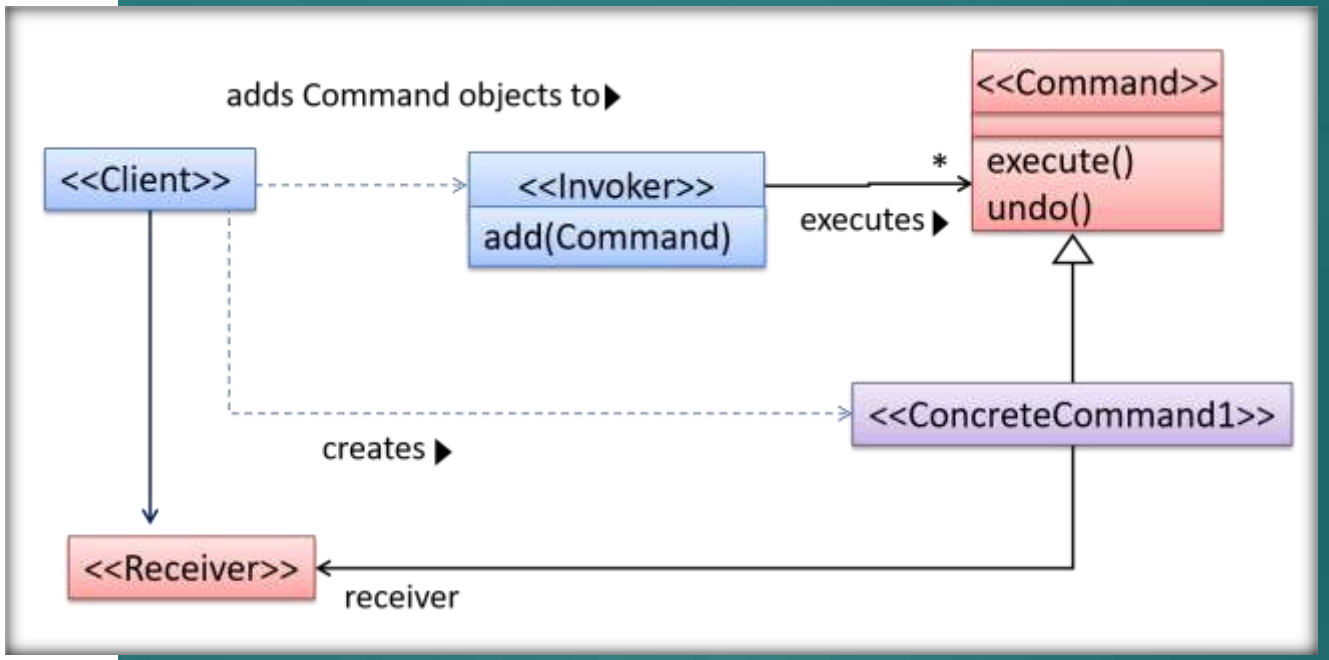


Have a look at the `W10→11-weekly-briefing-exercises.pdf` (in Canvas/files) if you haven't done so already.

Q2

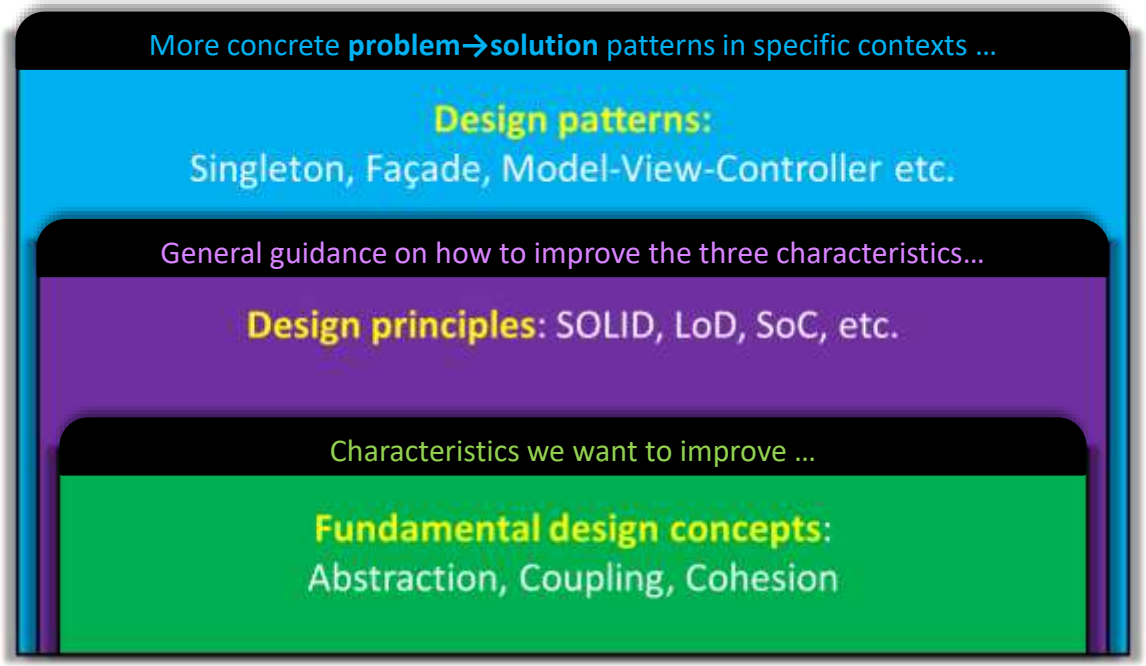
Do we have the Command pattern in AB3?





- ○ Yes
- ○ No

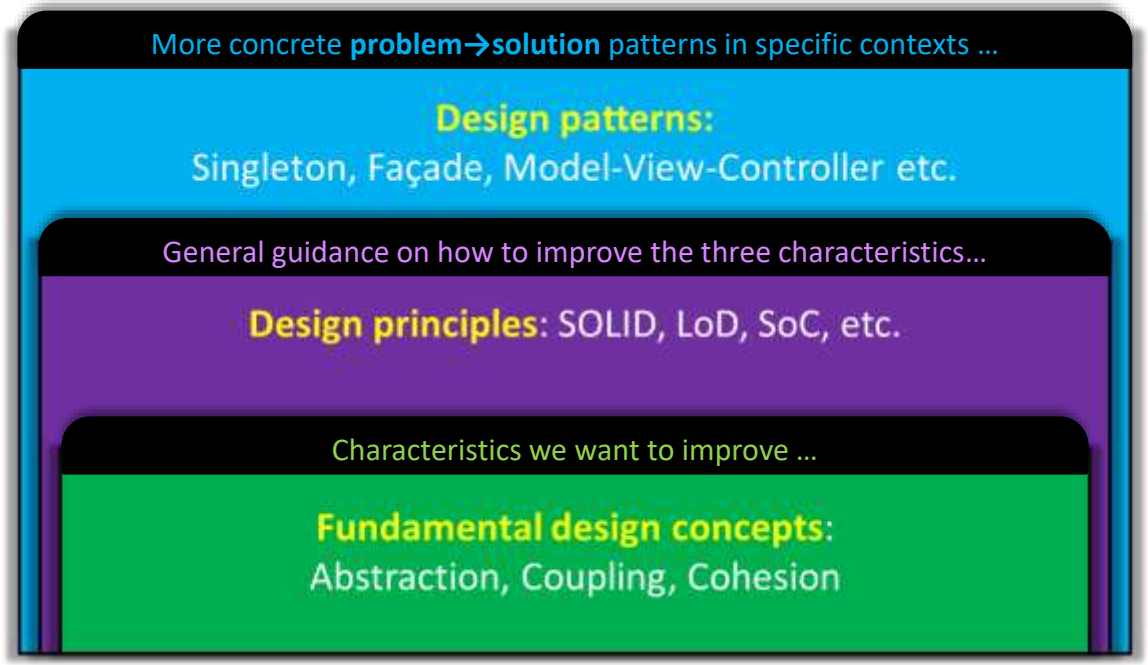
Previously, on the topic of **design** …



Week 11

Topics:

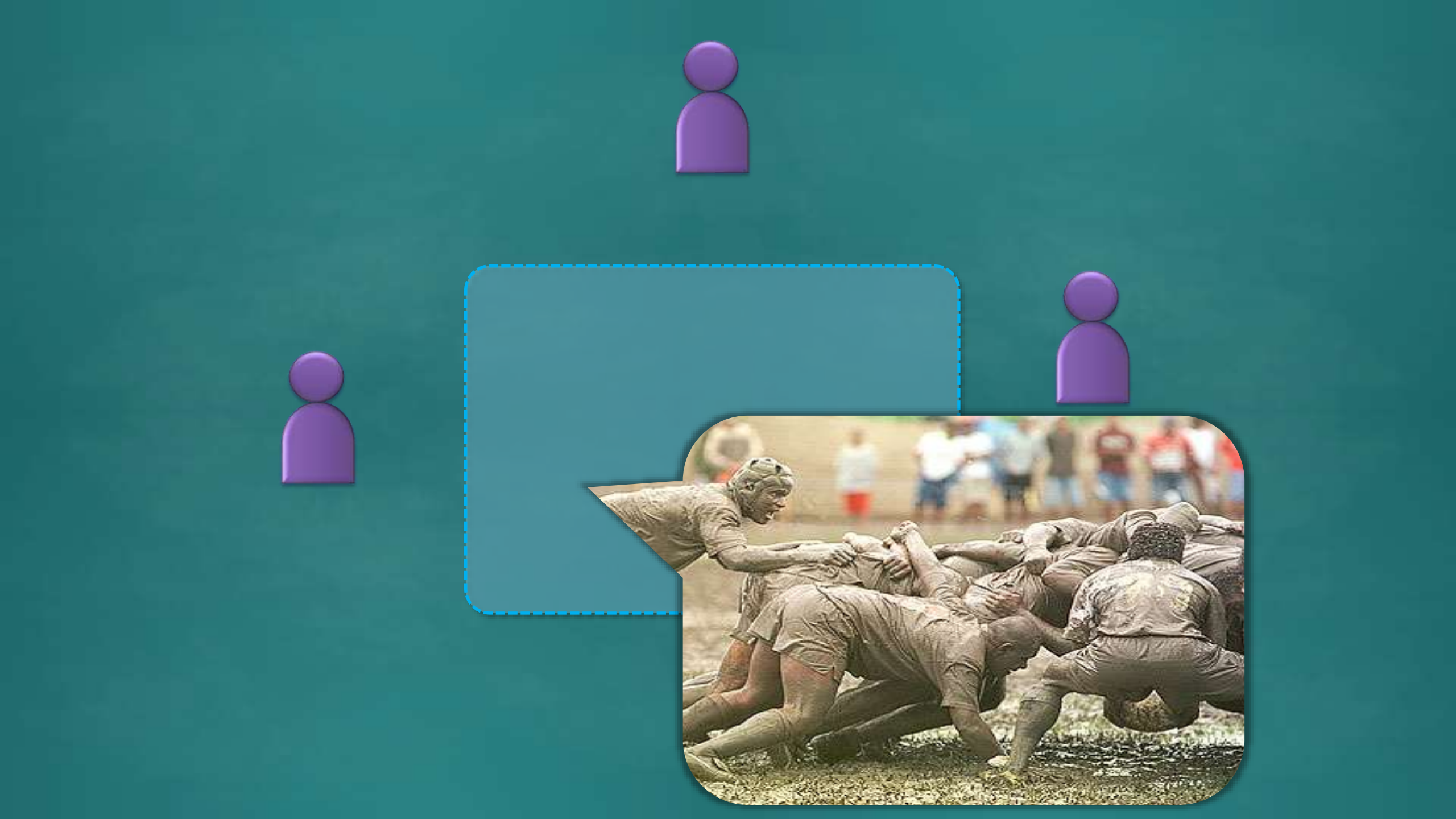
- [W11.1] **More Design Patterns** ⚡⚡
  - Singleton
  - Façade
  - Command
  - MVC
  - Observer
- [W11.2] Architectural Styles
- [W11.3] Test Cases: Inputs ⚡⚡
- [W11.4] Other QA Techniq
- [W11.5] Reuse ⚡
- [W11.6] Cloud Computing ⚡⚡⚡⚡: OPTIONAL
- [W11.7] Other UML Models ⚡⚡⚡⚡: OPTIONAL

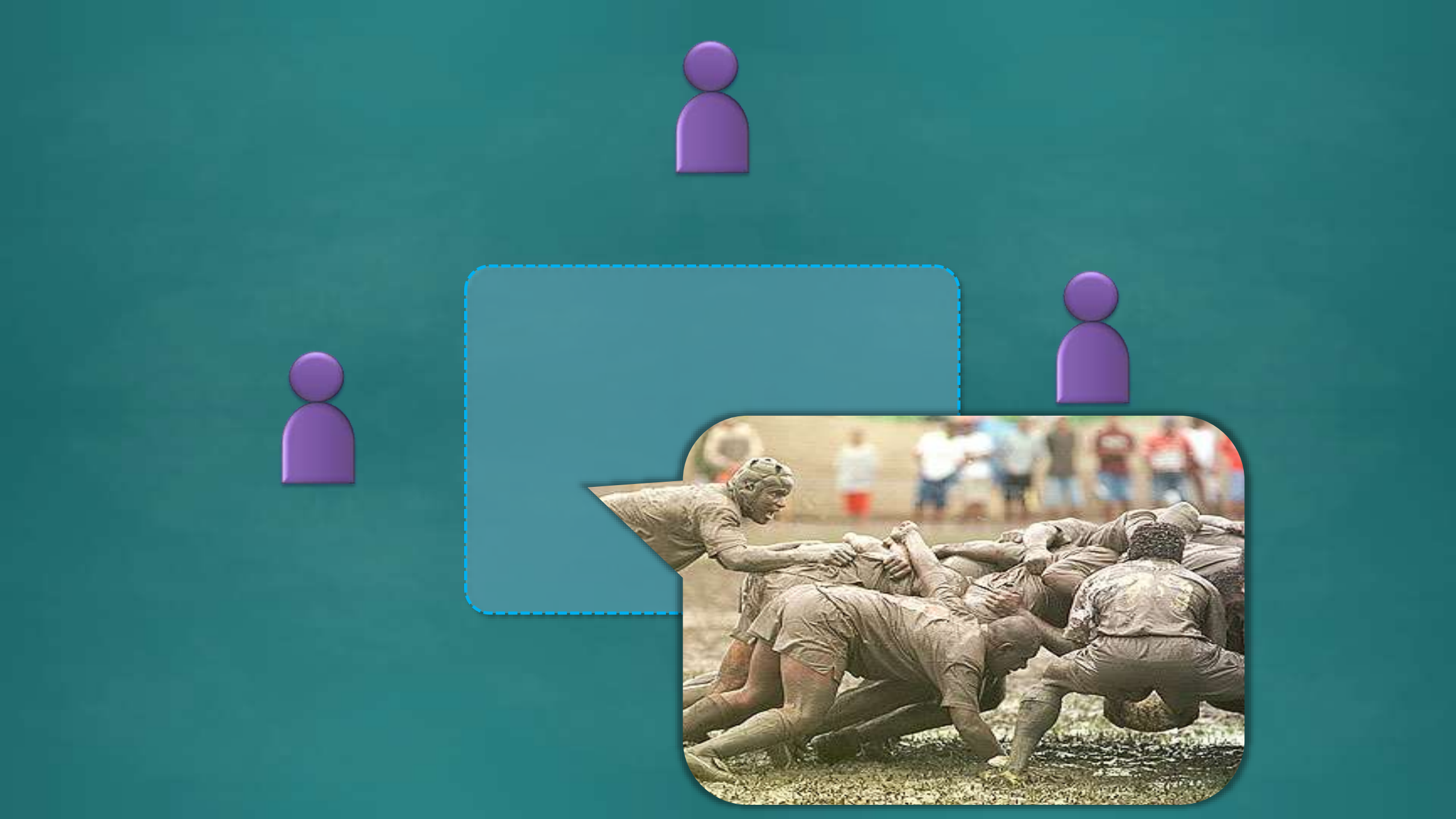


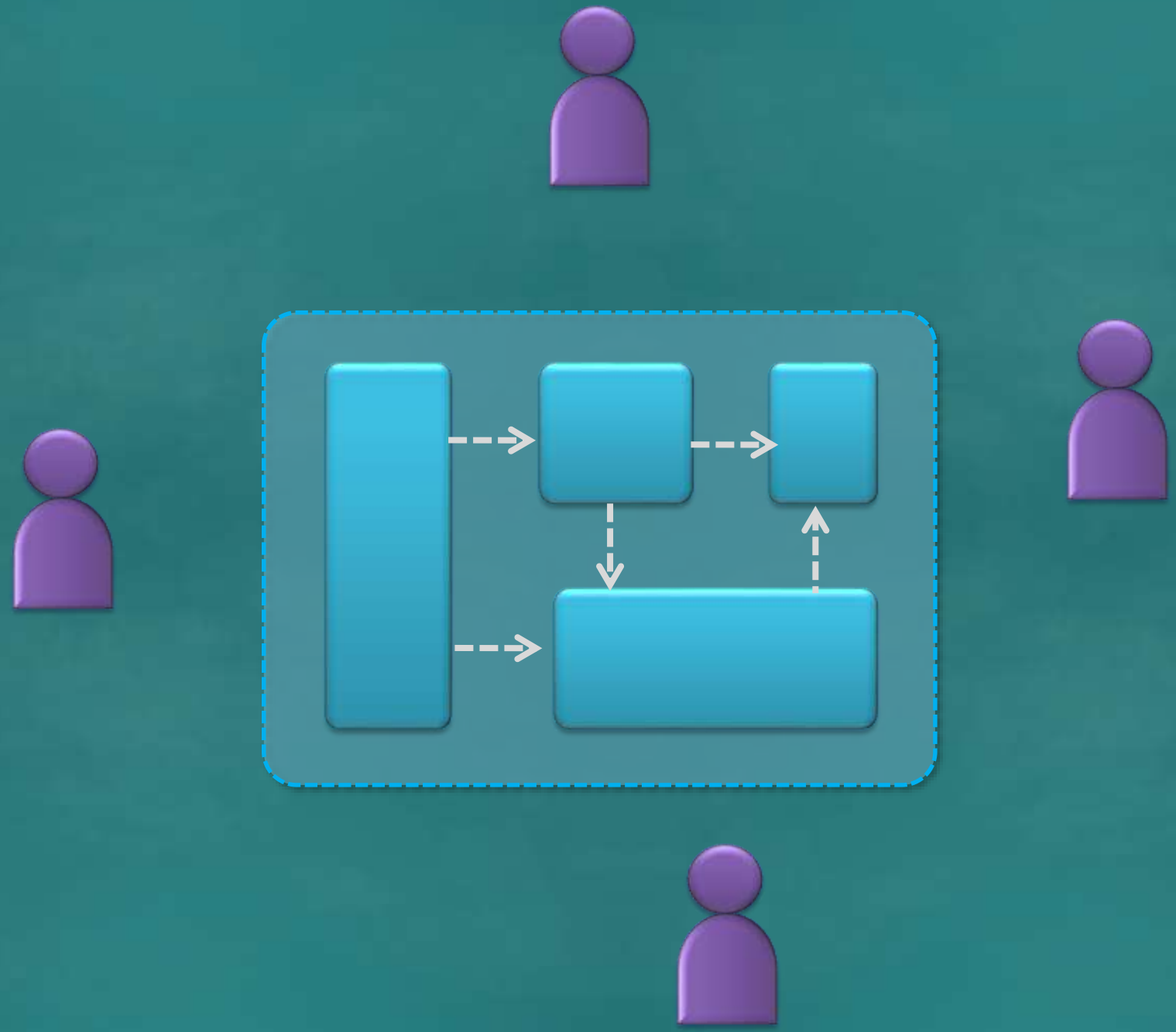

WAITING
WAITING
WAITING
WAITING
WAITING
WAITING
WAITING
WAITING
working


WAITING
WAITING
WAITING
WAITING
WAITING
WAITING
WAITING
working
slide added by
PowerPoint
Labs

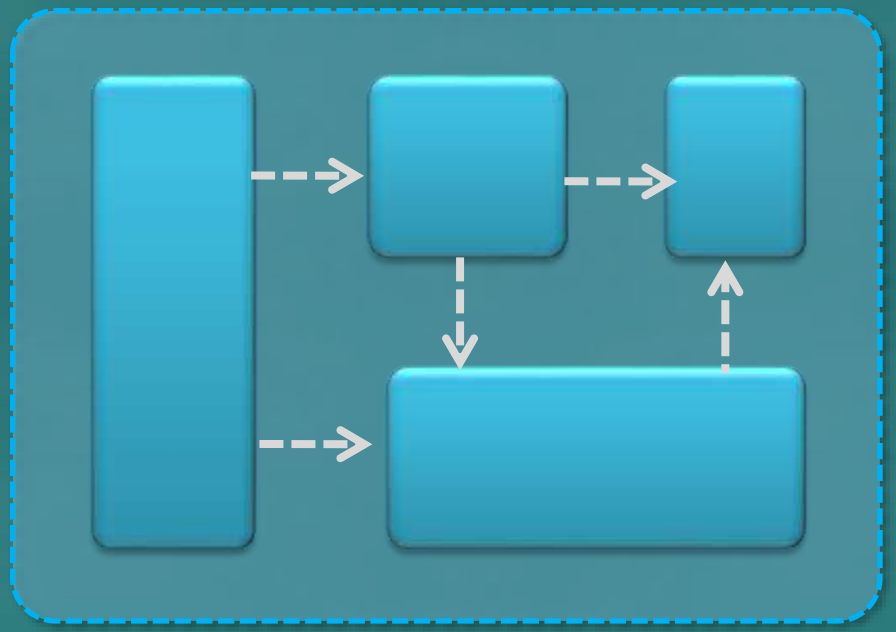

WAITING
WAITING
WAITING
WAITING
WAITING
WAITING
WAITING
working







Architecture



+ (important technical decisions)

Designed even before coding;
Should last the product life!

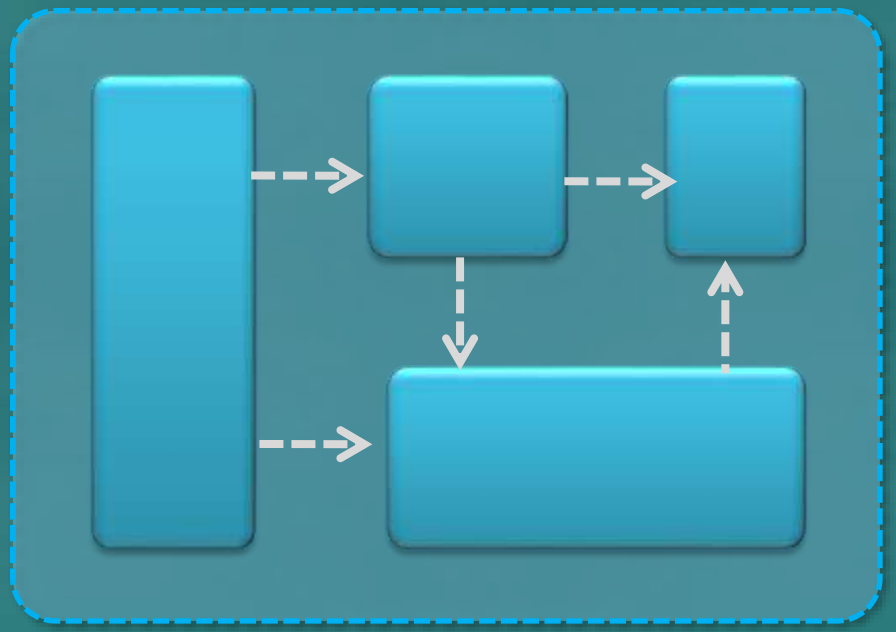
Who designs?

Programming skills not enough!
Ability to see things at a higher level is needed, among other things (e.g., use models).

	QUALIFICATIONS	EXPERIENCE (YEARS)	MIN	MAX
Software Engineer/ Senior Software Engineer	Degree	3-6	4,000	8,000
Software Technical Lead	Degree	5-8	5,700	8,000
Mobile Application Developer	Degree	1-3		6,000
Senior Mobile Application Developer	Degree	3-6		8,000
Java/ J2EE Software Engineer	Degree	3-5	3,500	6,000
Senior Java/ J2EE Software Engineer	Degree	5-10	6,000	10,000
Solution Architect	Degree	6-10	8,000	13,000
Application Support Analyst	Degree	2-6	3,000	6,500
System Analyst / Senior System Analyst	Degree	3-8	4,000	6,500
UI/ UX Designer	Degree	3-5	4,000	6,500
UI/ UX Lead Designer	Degree	6-10	7,000	12,000
QA Engineer/ Senior QA Engineer	Degree	5-10	5,000	8,000

Not automatic!

Architecture



+ (important technical decisions)

Who designs? Architect

Week 11

Topics:

- [W11.1] **More Design Patterns** ⚡⚡
- [W11.2] **Architectural Styles** ⚡⚡

Like design patterns, but for architecture …

- Client-server
- Event-driven
- Service-oriented
- …
- Microservices
- Agentic
- …

Admin:

1. Submit weekly quiz
2. Submit the PE mode selection
   COMPULSORY | Sat, Apr 5th 2359

tP: v1.5

1. Alpha-test the product
2. Fix alpha-test bugs, fine-tune features, improve code quality
3. Update UG and DG
4. Release v1.5 Thu, Apr 3rd 23:59
5. Attend the practical exam dry run
   Fri, Apr 4th 1600-1800

Q3

This is similar to an exam question. So, let's unpack it.

One of several testing types

Acceptance testing is most likely to be…

Done by users, to check if the product solves their problem

- … based on external behavior only ○ Black box testing
- … based on code ○ White box testing
- … a combination of the other two ○ Gray box testing

Three alternative test case design approaches

This is where you should spend most time!

~ 1-1.5 minutes

~ 2 minutes

In the exam, there will be 2nd part e.g.,

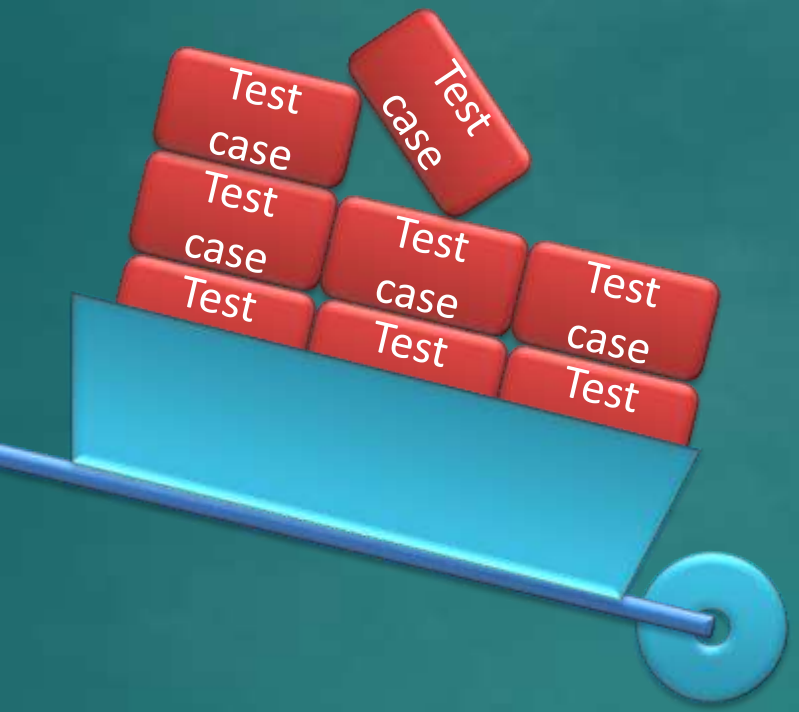
(b) Justify your choice.

```
No knowledge of code required.
```

Take away: you should be able to easily recall the **meaning of key concepts, and where they fit in**.

Why choose? → E&E of testing

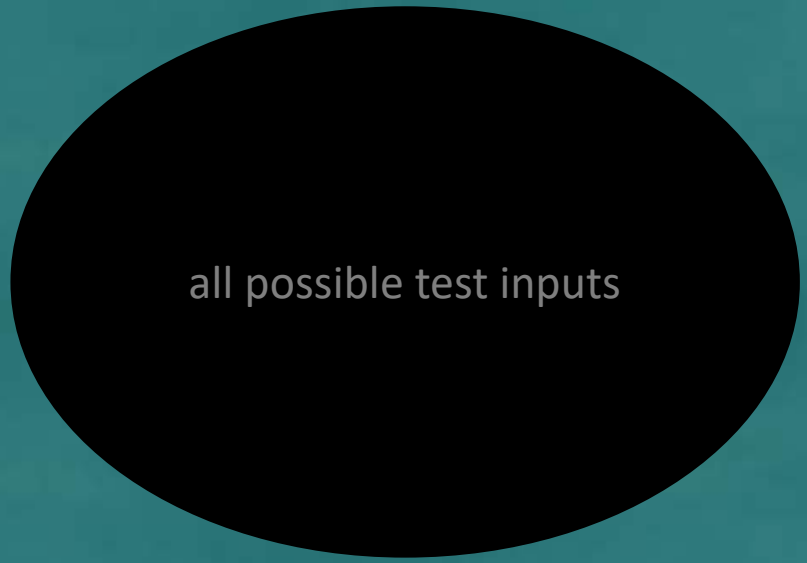
How to choose? → Some heuristics



Equivalence partitioning

Boundary value analysis

Equivalence partitioning



1. We can avoid testing too many cases from one partition
2. We can ensure all partitions are tested

Q

Equivalence partitioning

- ❑ increases efficiency of testing
- ❑ increases effectiveness of testing

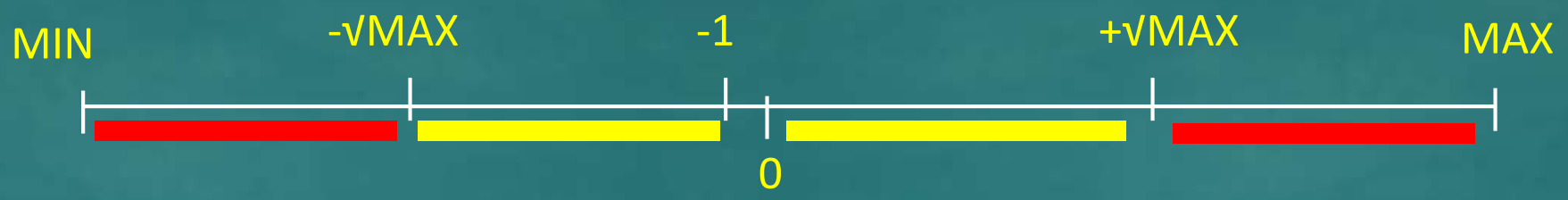
1. We can avoid testing too many cases from one partition
2. We can ensure all partitions are tested

Q

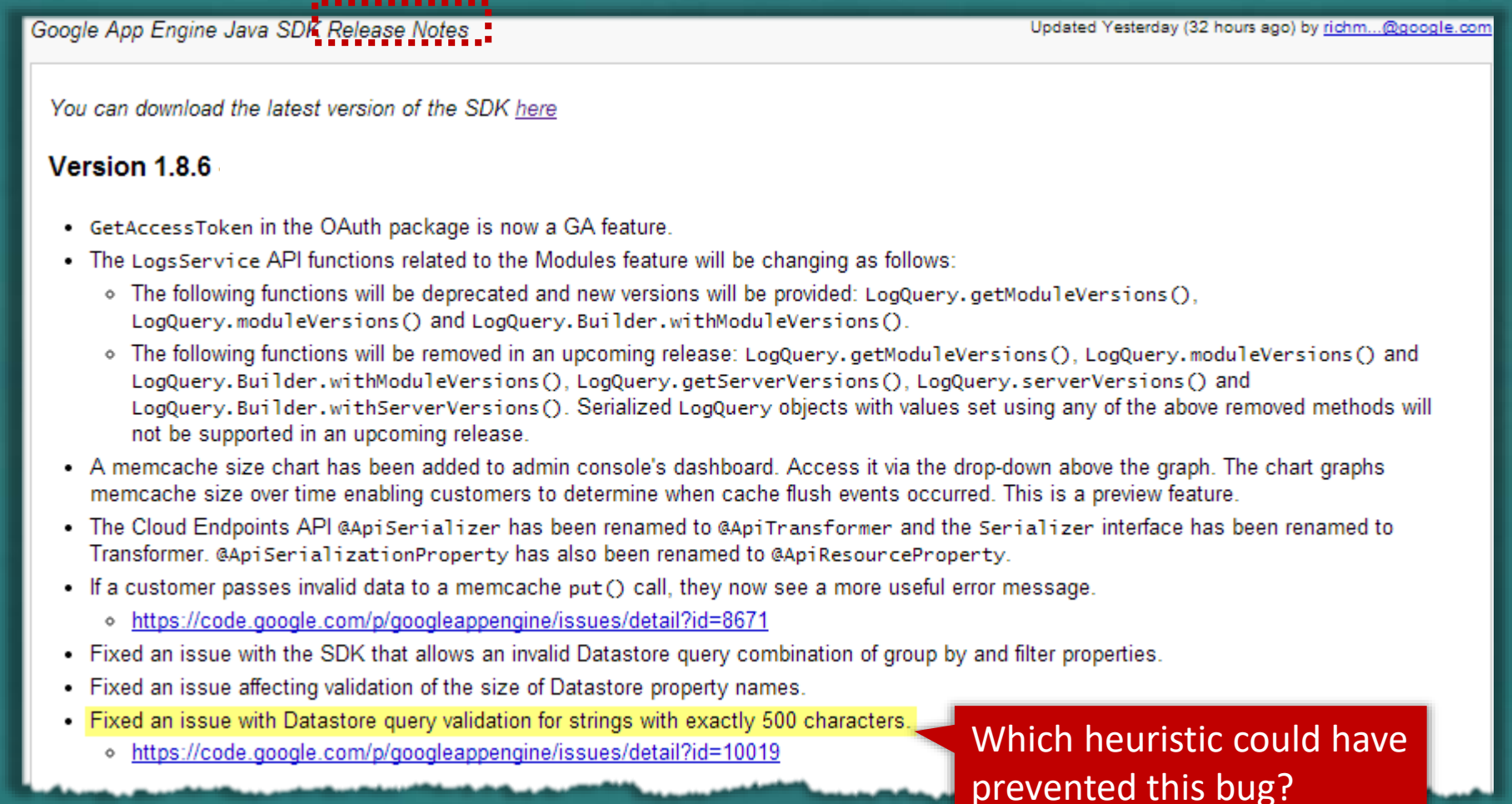
Does *i* need more than one eq. partition?

int square (int i)

Description: returns the the square value of *i*



- ○ Yes
- ○ No



Google App Engine Java SDK Release Notes

Updated Yesterday (32 hours ago) by richm...@google.com

You can download the latest version of the SDK here

Version 1.8.6

- `GetAccessToken` in the OAuth package is now a GA feature.
- The `LogsService` API functions related to the Modules feature will be changing as follows:
  - The following functions will be deprecated and new versions will be provided: `LogQuery.getModuleVersions()`, `LogQuery.moduleVersions()` and `LogQuery.Builder.withModuleVersions()`.
  - The following functions will be removed in an upcoming release: `LogQuery.getModuleVersions()`, `LogQuery.moduleVersions()` and `LogQuery.Builder.withModuleVersions()`, `LogQuery.getServerVersions()`, `LogQuery.serverVersions()` and `LogQuery.Builder.withServerVersions()`. Serialized `LogQuery` objects with values set using any of the above removed methods will not be supported in an upcoming release.
- A memcache size chart has been added to admin console's dashboard. Access it via the drop-down above the graph. The chart graphs memcache size over time enabling customers to determine when cache flush events occurred. This is a preview feature.
- The Cloud Endpoints API `@ApiSerializer` has been renamed to `@ApiTransformer` and the `Serializer` interface has been renamed to Transformer. `@ApiSerializationProperty` has also been renamed to `@ApiResourceProperty`.
- If a customer passes invalid data to a memcache `put()` call, they now see a more useful error message.
  - https://code.google.com/p/googleappengine/issues/detail?id=8671
- Fixed an issue with the SDK that allows an invalid Datastore query combination of group by and filter properties.
- Fixed an issue affecting validation of the size of Datastore property names.
- Fixed an issue with Datastore query validation for strings with exactly 500 characters.
  - https://code.google.com/p/googleappengine/issues/detail?id=10019

Which heuristic could have prevented this bug?

Boundary value analysis

```
public class StringUtilTest {

    //---------------- Tests for isNonZeroUnsignedInteger --------------------------------

    @Test
    public void isNonZeroUnsigne

        // EP: empty strings
        assertFalse(StringUtil.isNonZeroUnsignedInteger("")); // Boundary value
        assertFalse(StringUtil.isNonZeroUnsignedInteger("  "));

        // EP: not a number
        assertFalse(StringUtil.isNonZeroUnsignedInteger("a"));
        assertFalse(StringUtil.isNonZeroUnsignedInteger("aaa"));

        // EP: zero
        assertFalse(StringUtil.isNonZeroUnsignedInteger("0"));

        // EP: zero as prefix
        assertTrue(StringUtil.isNonZeroUnsignedInteger("01"));

        // EP: signed numbers
        assertFalse(StringUtil.isNonZeroUnsignedInteger("-1"));
```

Reason:

- For future devs' reference
- For grading

Q

When designing test cases in the tP …

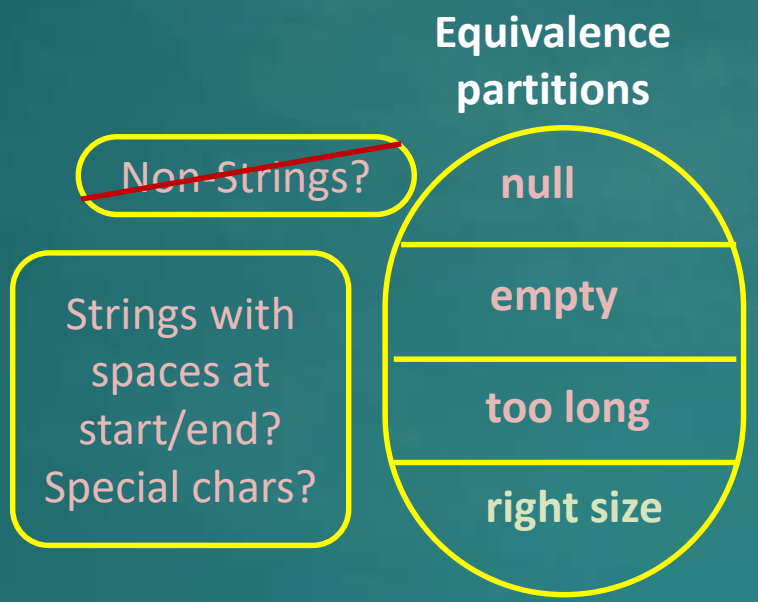
- ❑ A. apply heuristics when it helps.
- ❑ B. add comments in test cases to describe how a heuristic was used.
- ❑ C. every test case must strictly follow these heuristics.

Why?

- Heuristics might not apply for some cases.
- Heuristics-based test cases might not catch all bugs!

Q3 (a) Design test cases for *unit* testing the Parser#isValidName method below.

```
/* Returns true if the name is non-empty and
   not null and not longer than 40 chars. */

public static boolean isValidName(String name) {
    // ...
}
```



Values

null

“ ” “”

length = 41 , 50

40, 1, 10

Test input	Expected
null	False
“ ”	False
Length==41	False
Length==50	False
Length==40	True
Length==1	True

Q3 (a) Design test cases for *unit* testing the Parser#isValidName method below.

```
/* Returns true if the name is non-empty and
   not null and not longer than 40 chars. */

public static boolean isValidName(String name) {
    // ...
}
```

Equivalence partitions	Values
null	null
empty	“ ”
too long	length = 41 , 50
right size	40, 1

	Test input	Expected
[ ]	5 (int)	-
[ ]	null	False
[ ]	“ ”	False
[ ]	Length==41	False
[ ]	Length==50	False
[ ]	Length==40	True
[ ]	Length==1	True

```
/* Throws StorageException if name is not valid
   or if name already exists in the database. */

public void saveScore(String name)
                          throws StorageException {

    if (!Parser.isValidName(name)) {
      throw new StorageException("invalid name");
    }

    if (storage.isFound(name)) {
       throw new StorageException("already exists");
    }

    storage.save(name);
}
```

(b) Design *integration* test cases for the method below. Note that it uses the method given in (a).

How do we unit test this method?

If **Unit testing** , we should use stubs for dependencies `isValidName` etc.

```
/* Throws StorageException if name is not valid
   or if name already exists in the database. */

public void saveScore(String name)
                          throws StorageException {

    if (!Parser.isValidName(name)) {
      throw new StorageException("invalid name");
    }

    if (storage.isFound(name)) {
       throw new StorageException("already exists");
    }

    storage.save(name);
}
```

(b) Design *integration* test cases for the method below. Note that it uses the method given in (a).

Use all these test cases too?

Test input	Expected
null	False
“ ”	False
Length==41	False
Length==50	False
Length==40	True
Length==1	True

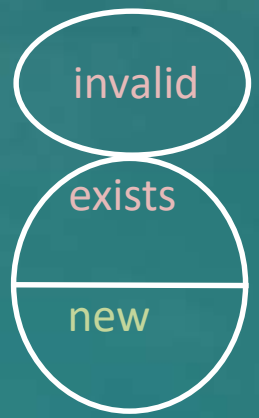
Integration tests are choreography tests.
They do not test components.
They test how well the assembly of components dance together.

```java
/* Throws StorageException if name is not valid
   or if name already exists in the database. */

public void saveScore(String name)
                          throws StorageException {

    if (!Parser.isValidName(name)) {
      throw new StorageException("invalid name");
    }
    //more code (does not involve dependencies)
    if (storage.isFound(name)) {
       throw new StorageException("already exists");
    }
    //more code (does not involve dependencies)
    storage.save(name);
}
```

(b) Design *integration* test cases for the method below. Note that it uses the method given in (a).

To be tested by unit tests

Test input	Expected
"new guy"	success
null	exception
"existing guy"	exception

Eq. partitions	Values
	Any invalid e.g. null Any existing name e.g. "existing guy" Any valid new name e.g. "new guy"

Q

```
/* Throws StorageException if name is not valid
   or if name already exists in the database. */

public void saveScore(String name)
                            throws StorageException {

    if (!Parser.isValidName(name)) {
      throw new StorageException("invalid name");
    }

    if (storage.isFound(name)) {
       throw new StorageException("already exists");
    }

    storage.save(name);
}
```

How many test cases?

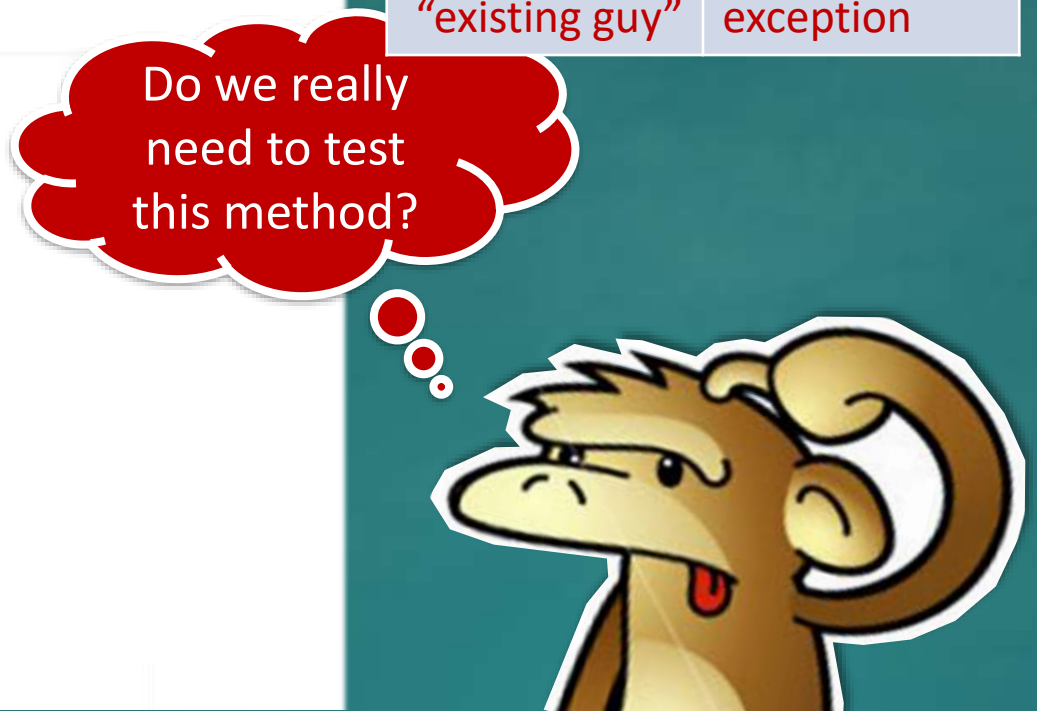
- 2
- 3
- 4
- 5

Test input	Expected
“new guy”	success
null	exception
“existing guy”	exception

Eq. partitions	Values
invalid	Any invalid e.g. null
exists	Any existing name e.g. “existing guy”
new	Any valid new name e.g. “new guy”

```
/* Throws StorageException if name is not valid
   or if name already exists in the database. */

public void saveScore(String name)
                            throws StorageException {

    if (!Parser.isValidName(name)) {
      throw new StorageException("invalid name");
    }

    if (storage.isFound(name)) {
       throw new StorageException("already exists");
    }

    storage.save(name);
}
```

Test input	Expected
“new guy”	success
null	exception
“existing guy”	exception



```
/* Throws StorageException if name is not valid
   or if name already exists in the database. */

public void saveScore(String name)
                            throws StorageException {

    if (!Parser.isValidName(name)) {
      throw new StorageException("invalid name");
    }

    if (storage.isFound(name)) {
       throw new StorageException("already exists");
    }

    storage.save(name);
}
```

Test input	Expected
“new guy”	success
null	exception
“existing guy”	exception

Even a stopped clock looks correct twice a day!

Do we really need to test this method?

Q

```
/* Throws StorageException if name is not valid
   or if name already exists in the database. */

public void saveScore(String name)
                            throws StorageException {

    if (!Parser.isValidName(name)) {
      throw new StorageException("invalid name");
    }

    if (storage.isFound(name)) {
       throw new StorageException("already exists");
    }

    storage.save(name);
}
```

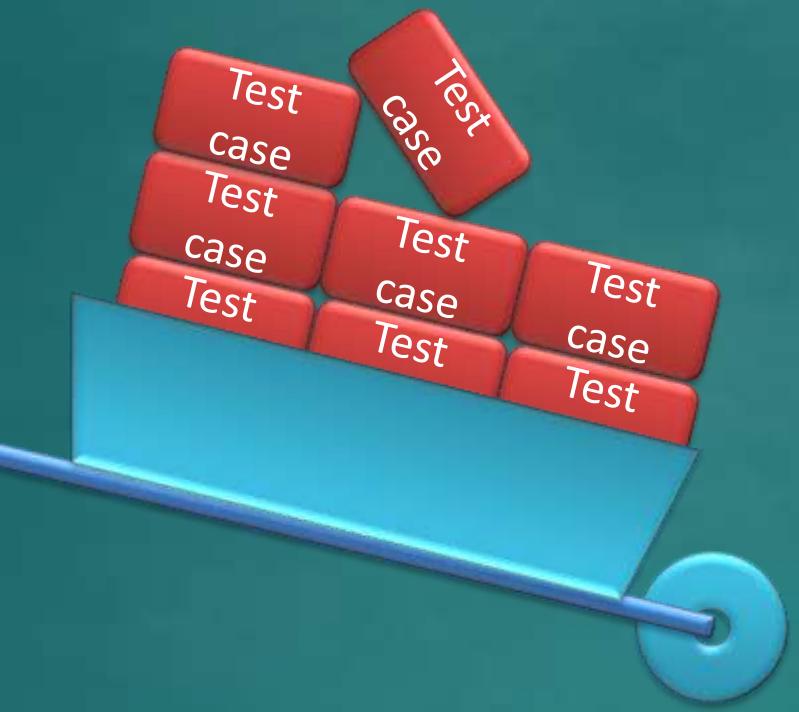
Test input	Expected
“new guy”	success
null	exception
“existing guy”	exception

Do we really need to test this method?

- ○ Yes we must.
- ○ No need. Too trivial.
- ○ Good to, if not too costly.

Why choose? → E&E of testing

How to choose? → Some heuristics



Equivalence partitioning

Boundary value analysis

Combining multiple inputs

What if there are multiple inputs to the SUT?

Week 11

Topics:

- [W11.1] **More Design Patterns** ⚡⚡
- [W11.2] **Architectural Styles** ⚡⚡
- [W11.3] **Test Cases: Combining Multiple Inputs** ⚡⚡
- [W11.4] **Other QA Techniques** ⚡⚡
- [W11.5] **Reuse** ⚡
- [W11.6] **Cloud Computing** ⚡⚡⚡⚡: OPTIONAL
- [W11.7] **Other UML Models** ⚡⚡⚡⚡: OPTIONAL

Admin:

1. Submit weekly quiz (P)
2. Submit the PE mode selection
   COMPULSORY | Sat, Apr 5th 2359

tP: v1.5

1. Alpha-test the product
2. Fix alpha-test bugs, fine-tune features, improve code quality
3. Update UG and DG
4. Release v1.5 Thu, Apr 3rd 23:59
5. Attend the practical exam dry run
   Fri, Apr 4th 1600-1800 (P)

Q2. Suggest at least 5 ways (no more than 2 related to the coding standard) to improve the quality of the following code.

```
1  /* Set user as ‘active’ on the server.
2   * @throws CannotActivateException if the user doesn’t exist on the server
3   */
4  void activateUserOnServer(String userName) throws CannotActivateException{

5      log("trying to activate");

6      assert isUsernameAcceptable(userName);

7      ServerConnection.activate(userName);
8      if(!ServerConnection.isActivated(userName))
9          throw new CannotActivateException(userName + " not activated");



10     Account account = AccountManager.getAccount(userName);
11     account.toggleActivatedStatus(); //mark as activated
12 }
```

Q

Fixing coding standard violations:

```
1  /* Set user as ‘active’ on the server.
2   * @throws CannotActivateException if the user doesn’t exist on the server
3   */
4  void activateUserOnServer(String userName) throws CannotActivateException{

5      log("trying to activate");

6      assert isUsernameAcceptable(userName);

7      ServerConnection.activate(userName);
8      if(!ServerConnection.isActivated(userName))
9          throw new CannotActivateException(userName + " not activated");



10     Account account = AccountManager.getAccount(userName);
11     account.toggleActivatedStatus(); //mark as activated
12 }
```

- ☑ (a) Line 1: /* → /**
- ☑ (b) Line 1: Set → Sets
- x ☐ (c) Line 4: Add missing visibility
- x ☐ (d) Line 5: The log message can be more descriptive
- ☑ (e) Line 8: Add missing braces

Q

Any problems w.r.t. assertions, exceptions?

```
1  /* Set user as 'active' on the server.
2   * @throws CannotActivateException if the user doesn't exist on the server
3   */
4  void activateUserOnServer(String userName) throws CannotActivateException{

5      log("trying to activate");

6      assert isUsernameAcceptable(userName);

7      ServerConnection.activate(userName);
8      if(!ServerConnection.isActivated(userName))
9          throw new CannotActivateException(userName + " not activated");



10     Account account = AccountManager.getAccount(userName);
11     account.toggleActivatedStatus(); //mark as activated
12 }
```

- ☑ (a) Line 6: Undocumented assumption or should use an exception
- ☑ (b) Line 7-9: Behavior doesn't match the advertised behavior

Q

Any problems w.r.t. coupling and cohesion?

```
1  /* Set user as 'active' on the server.
2   * @throws CannotActivateException if the user doesn't exist on the server
3   */
4  void activateUserOnServer(String userName) throws CannotActivateException{

5      log("trying to activate");

6      assert isUsernameAcceptable(userName);

7      ServerConnection.activate(userName);
8      if(!ServerConnection.isActivated(userName))
9          throw new CannotActivateException(userName + " not activated");



10     Account account = AccountManager.getAccount(userName);
11     account.toggleActivatedStatus(); //mark as activated
12 }
```

- ☑ (a) Lines 10-11: Reduces cohesion
- ☑ (b) Lines 10-11: Increases coupling
- x ☐ (c) Lines 10-11: Behavior not advertised (hidden behavior)

Q

Any further problems w.r.t. defensive coding?

```
1  /* Set user as ‘active’ on the server.
2   * @throws CannotActivateException if the user doesn’t exist on the server
3   */
4  void activateUserOnServer(String userName) throws CannotActivateException{

5      log("trying to activate");

6      assert isUsernameAcceptable(userName);

7      ServerConnection.activate(userName);
8      if(!ServerConnection.isActivated(userName))
9          throw new CannotActivateException(userName + " not activated");



10     Account account = AccountManager.getAccount(userName);
11     account.toggleActivatedStatus(); //mark as activated
12 }
```

- ☐ (a) Line 6: Not defensive
- ☑ (b) Line 11: Not defensive

Whoa! So many things to consider in so little code!

```
1  /* Set user as 'active' on the server.
2   * @throws CannotActivateException if the user doesn't exist on the server
3   */
4  void activateUserOnServer(String userName) throws CannotActivateException{

5      log("trying to activate");

6      assert isUsernameAcceptable(userName);

7      ServerConnection.activate(userName);
8      if(!ServerConnection.isActivated(userName))
9          throw new CannotActivateException(userName + " not activated");



10     Account account = AccountManager.getAccount(userName);
11     account.toggleActivatedStatus(); //mark as activated
12 }
```

It's OK if you didn't think of all those code quality problems.

- Some are not obvious, some are subjective.
- There may be others not mentioned.

Good enough if you were able to understand the discussion.

Week 11

Topics:

- [W11.1] **More Design Patterns** ⚡⚡
- [W11.2] **Architectural Styles** ⚡⚡
- [W11.3] **Test Cases: Combining Multiple Inputs** ⚡⚡
- [W11.4] **Other QA Techniques** ⚡⚡
- [W11.5] **Reuse** ⚡
- [W11.6] **Cloud Comput**
- [W11.7] **Other UML Mo**

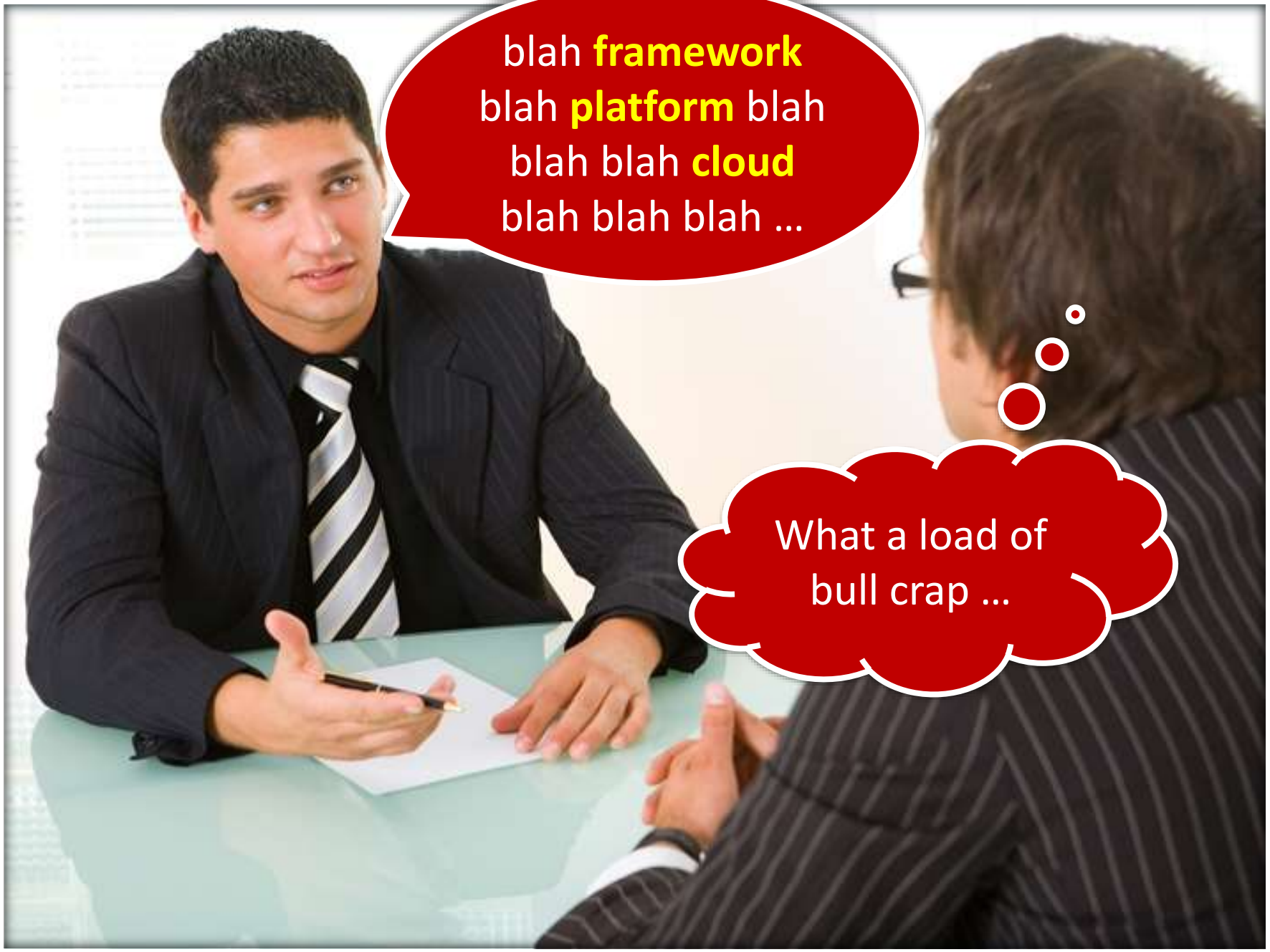
QA
- Testing
- Code reviews
- Static analysis
- Formal methods

Admin:

1. Submit weekly quiz
2. Submit the PE mode selection
   COMPULSORY | Sat, Apr 5th 2359

tP: v1.5

1. Alpha-test the product
2. Fix alpha-test bugs, fine-tune features, improve code quality
3. Update UG and DG
4. Release v1.5 Thu, Apr 3rd 23:59
5. Attend the practical exam dry run
   Fri, Apr 4th 1600-1800


blah framework
blah platform blah
blah blah cloud
blah blah blah …
What a load of
bull crap …

Week 11

Topics:

- [W11.1] **More Design Patterns** ⚡⚡
- [W11.2] **Architectural Styles** ⚡⚡
- [W11.3] **Test Cases: Combining Multiple Inputs** ⚡⚡
- [W11.4] **Other QA Techniques** ⚡⚡
- [W11.5] **Reuse** ⚡
- [W11.6] **Cloud Computing** ⚡⚡⚡⚡: OPTIONAL
- [W11.7] **Other UML Models** ⚡⚡⚡⚡: OPTIONAL

Admin:

1. Submit weekly quiz
2. Submit the PE mode selection
   COMPULSORY | Sat, Apr 5th 2359

tP: v1.5

1. Alpha-test the product
2. Fix alpha-test bugs, fine-tune features, improve code quality
3. Update UG and DG
4. Release v1.5 Thu, Apr 3rd 23:59
5. Attend the practical exam dry run
   Fri, Apr 4th 1600-1800

Week 11

Topics:

- [W11.1] **More Design Patterns** ⚡⚡
- [W11.2] **Architectural Styles** ⚡⚡
- [W11.3] **Test Cases: Combining Multiple Inputs** ⚡⚡
- [W11.4] **Other QA Techniques** ⚡⚡
- [W11.5] **Reuse** ⚡
- [W11.6] **Cloud Computing** ⚡⚡⚡⚡: OPTIONAL
- [W11.7] **Other UML Models** ⚡⚡⚡⚡: OPTIONAL

Admin:

1. Submit weekly quiz
2. Submit the PE mode selection
   COMPULSORY | Sat, Apr 5th 2359

tP: v1.5

1. Alpha-test the product
2. Fix alpha-test bugs, fine-tune features, improve code quality
3. Update UG and DG
4. Release v1.5 Thu, Apr 3rd 23:59
5. Attend the practical exam dry run
   Fri, Apr 4th 1600-1800

Week 11

Topics:

- [W11.1] **More Design Patterns** ⚡⚡
- [W11.2] **Architectural Styles** ⚡⚡
- [W11.3] **Test Cases: Combining Multiple Inputs** ⚡⚡
- [W11.4] **Other QA Techniques** ⚡⚡
- [W11.5] **Reuse** ⚡
- [W11.6] **Cloud Computing** ⚡⚡⚡⚡: OPTIONAL
- [W11.7] **Other UML Models** ⚡⚡⚡⚡: OPTIONAL

Admin:

1. Submit weekly quiz (P)
2. Submit the PE mode selection
   COMPULSORY | Sat, Apr 5th 2359

tP: v1.5

1. Alpha-test the product
2. Fix alpha-test bugs, fine-tune features, improve code quality
3. Update UG and DG
4. Release v1.5 Thu, Apr 3rd 23:59
5. Attend the practical exam dry run
   Fri, Apr 4th 1600-1800 (P)

There is a lot to do!